Parallel Programming
NTU, Fall 2025, Group 27
Final Project Report✨

Our GitHub repository (including the reference repository):
https://github.com/ycli219/Parallel-Programming-Image-Color-Quantization-using-K-Means

Introduction

Image color quantization is a fundamental process in computer graphics and image processing, designed to **reduce the number of distinct colors in an image while preserving its visual essence**. This technique is crucial for compression, memory reduction, and artistic stylization. A highly effective method for achieving this is the **K-means clustering algorithm**, which treats individual pixels as 3D data points in the Red-Green-Blue (RGB) color space. The algorithm operates iteratively, identifying K dominant centroids and reassigning every pixel to its nearest centroid to approximate the original image.

However, **the computational cost of K-means presents a significant challenge**. The process involves calculating the Euclidean distance between every pixel and every cluster centroid during each iteration. For high-resolution images, the time complexity scales as **O(N * K * I)**, where N is the number of pixels, K is the number of clusters, and I is the number of iterations. This creates a substantial performance bottleneck in sequential execution, especially as the image resolution increases. Fortunately, the "Assignment Step"—where pixel distances are calculated—is inherently independent for each pixel, making the algorithm "pleasantly parallel" and an ideal candidate for acceleration.

The objective of this project is to implement, evaluate, and compare four distinct versions of the K-means algorithm: a **serial** baseline, a multi-core CPU version using **OpenMP**, a distributed-memory version using **MPI**, and a massively parallel GPU version using **CUDA**. By exploring these architectures, we aim to analyze the performance speedup and scalability characteristics of each parallel paradigm.

Method

The core K-means algorithm consists of two primary phases repeated until convergence or a set number of iterations:

1. **Assignment Step**: Each pixel is compared to all K centroids and assigned the label of the nearest one based on Euclidean distance.
2. **Update Step**: New centroids are calculated by computing the average color (RGB mean) of all pixels currently assigned to that cluster.

We implemented this logic across four different computing environments using C++ and OpenCV for image I/O:

1. **Serial Implementation (Baseline)**

   To establish a reliable performance baseline, we utilized an open-source single-threaded C++ implementation obtained from a public GitHub repository. This code utilizes `std::vector` to store `Pixel` objects and OpenCV `cv::Mat` structures for image manipulation.

   - **Initialization**: Centroids are initialized by **randomly selecting** K pixels from the source image.
   - **Execution**: The algorithm runs sequentially on the CPU. The distance calculation and centroid averaging are performed in a nested loop structure, iterating over rows and columns of the image one pixel at a time. This external implementation serves as the primary **performance baseline** against which the speedup of our parallel implementations is measured.

2. **Shared-Memory Parallelism (OpenMP)**

   For the multi-core CPU implementation, we developed a shared-memory parallel version using the OpenMP API to exploit thread-level parallelism on a single node. We focused on parallelizing the computationally intensive loops found in both the assignment and update phases.

   - **Loop Collapsing**: We applied the `#pragma omp parallel for collapse(2)` directive to the nested loops (rows and columns) of the image. This effectively flattens the loop iteration space, allowing OpenMP to distribute the workload evenly across available threads.
   - **Reduction**: In the `computeCentroids` function, we utilized the `reduction` clause (`+:mean_b, mean_g, mean_r, n`). This allows threads to compute local sums of pixel colors for each cluster in parallel without race conditions, merging them into a global sum at the end of the parallel region.

3. **Distributed-Memory Parallelism (MPI)**

   To evaluate the algorithm within a distributed-memory paradigm, we implemented a version using the Message Passing Interface (MPI). This approach relies on domain decomposition, where the image data is partitioned among multiple independent processes with isolated memory spaces.

   - **Data Distribution (Scatter)**: The root process (Rank 0) reads the image and broadcasts metadata (dimensions, K). We utilized `MPI_Scatterv` to physically distribute the raw pixel data to separate memory buffers in each process. This function was specifically chosen to handle cases where the total number of

pixels is not perfectly divisible by the number of processes, ensuring robust data partitioning.

- **Local Computation**: Each MPI process operates independently on its local data subset, performing the assignment step and accumulating local partial sums. Since memory is not shared, this simulates a scalable environment where computation is decoupled from global state.
- **Global Reduction (Allreduce)**: The critical synchronization point occurs during the centroid update. We employed `MPI_Allreduce` to aggregate partial color accumulators and counters from all processes. This ensures that every process receives the updated global centroids simultaneously via message passing before the next iteration.
- **Result Gathering**: Finally, `MPI_Gatherv` is used to collect the processed pixel data from all distributed buffers back to Rank 0 for final reconstruction.

4. **Massively Parallel Implementation (CUDA)**

The GPU implementation was designed to leverage the massive throughput of CUDA cores. This version required explicit memory management and architectural optimizations to minimize latency.

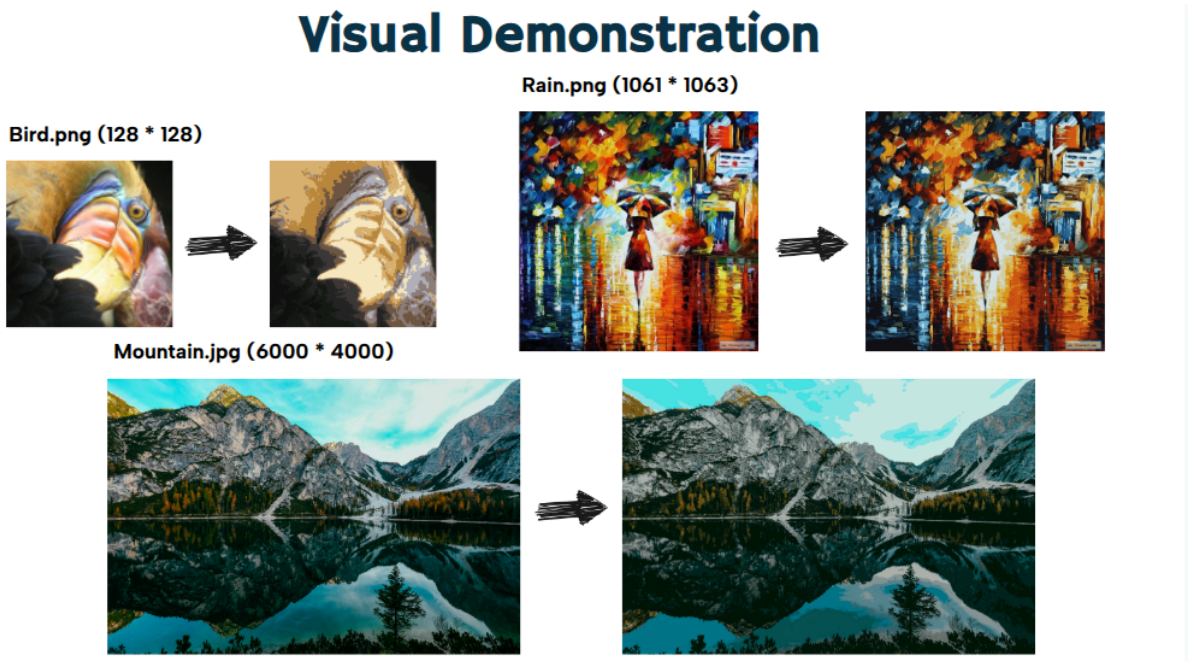
- **Constant Memory**: We stored the cluster centroids in the GPU's Constant Memory (`__constant__`). Since centroids are read-only during the assignment kernel and accessed frequently by every thread, this takes advantage of the GPU's constant cache to significantly reduce memory bandwidth pressure.
- **Shared Memory Optimization**: A naive implementation using global atomics for centroid updates is slow due to high contention. Instead, we implemented a **block-level reduction**. Threads within a block accumulate pixel values into Shared

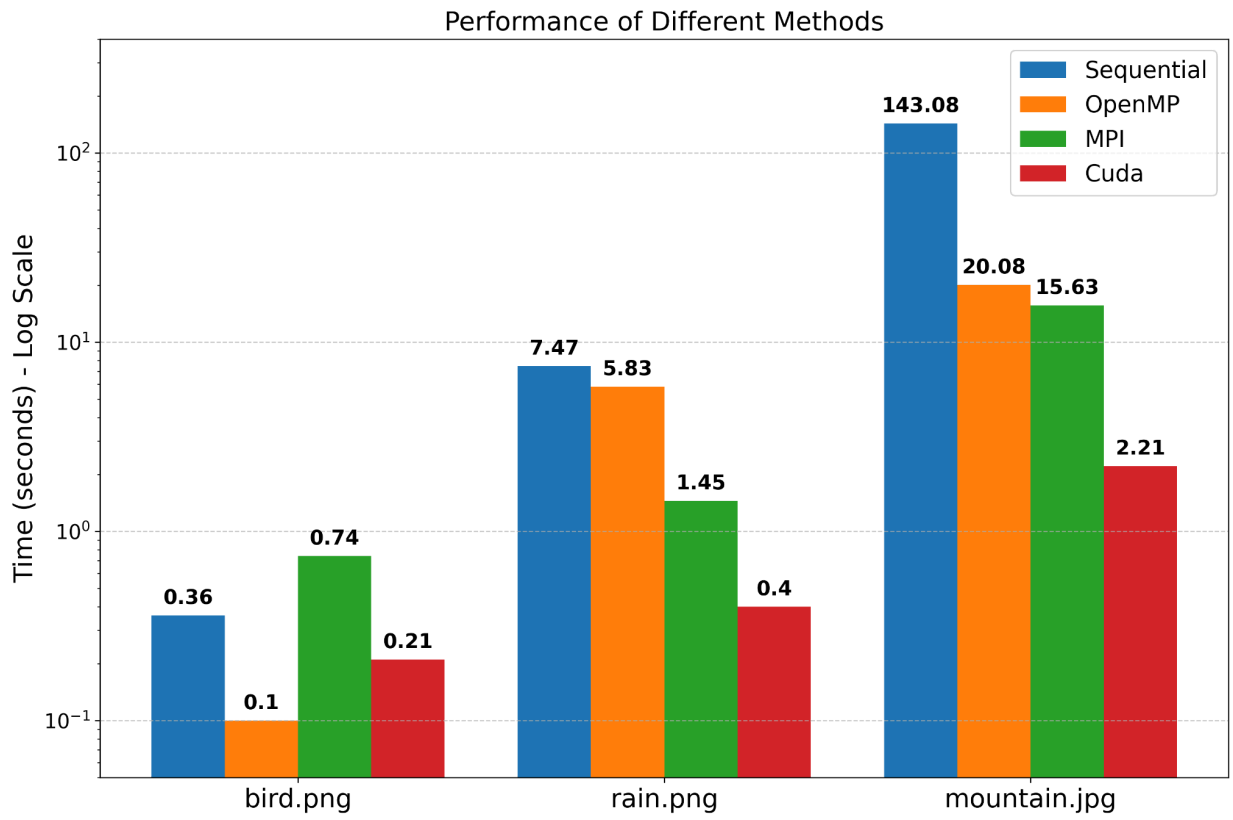
Memory (`__shared__`) first. Once the block finishes, only the aggregated block results are written to global memory arrays (`d_partial_sums`).

- **Two-Stage Kernel Approach**:
  I. **Assignment & Partial Reduction Kernel**: Computes pixel-to-centroid distances and accumulates partial sums into shared memory, then global memory.
  II. **Centroid Update Kernel**: A separate kernel (running with 1 block) gathers the partial sums from all blocks to compute the final averages. This keeps the data entirely on the GPU during the training loop, avoiding expensive Host-to-Device data transfers until the final image is ready.

Experiment - Overall

Dataset: bird.png, rain.png, mountain.jpg (see GitHub repo).





We compared the execution time of all four implementations across three datasets of varying resolution: `bird.png` (Small), `rain.png` (Medium), and `mountain.jpg` (Large). Note that the Y-axis is presented on a **logarithmic scale** to accommodate the significant performance differences.

1. **GPU Dominance on Large Scale**: The CUDA implementation consistently achieved the highest throughput on substantial workloads. On the largest dataset (`mountain.jpg`), CUDA reduced the execution time from **143.08s** (Sequential) to just **2.21s**, achieving a massive **~65x speedup**. This confirms that the massively parallel architecture of the GPU is ideally suited for the pixel-independent calculations of K-means when the workload is sufficient to saturate the device.

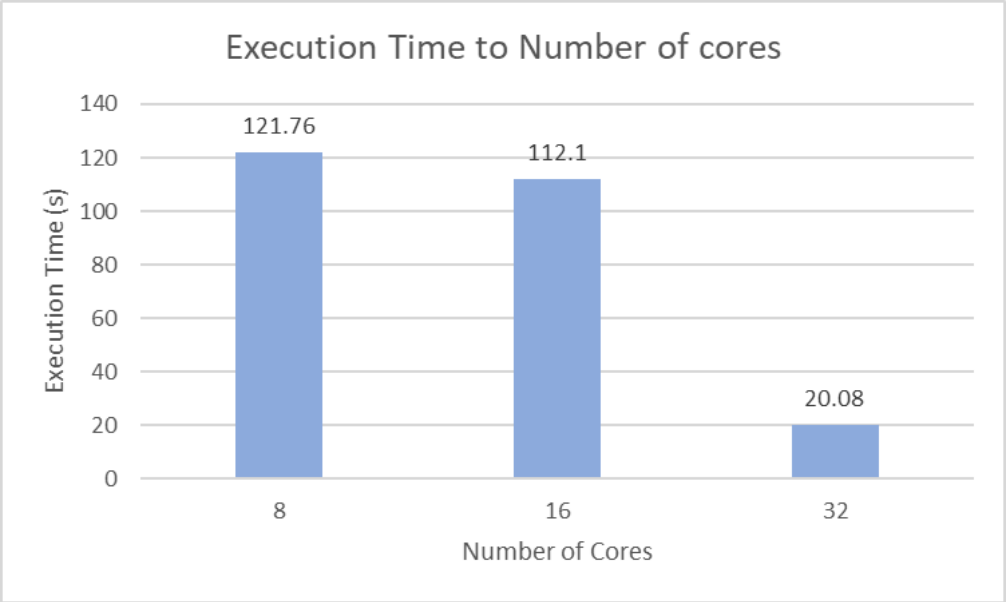
2. **The Impact of Overhead on Small Workloads**: The smallest dataset (`bird.png`) reveals the critical trade-offs involving initialization and data transfer costs.
    - **MPI vs. Sequential**: The MPI implementation (0.74s) performed worse than the Sequential baseline (0.36s), illustrating that message-passing latency outweighs computational gains for small N.
    - **CUDA vs. OpenMP**: Notably, **OpenMP (0.10s) outperformed CUDA (0.21s)** in this specific case. This is because the fixed overhead of transferring data over the PCIe bus (`cudaMemcpy`) and kernel launch latency is significant relative to the short computation time. The CPU (OpenMP) avoids this transfer cost and processes the small dataset efficiently entirely within its L3 cache.

3. **Scalability Trends**: As the image resolution increases (moving to `rain.png` and `mountain.jpg`), the overheads become negligible compared to the computation time. Both OpenMP and MPI demonstrate strong scalability, effectively amortizing their management costs. However, they remain an order of magnitude slower than the GPU solution, which thrives as the data size grows.

Experiment - Scalability

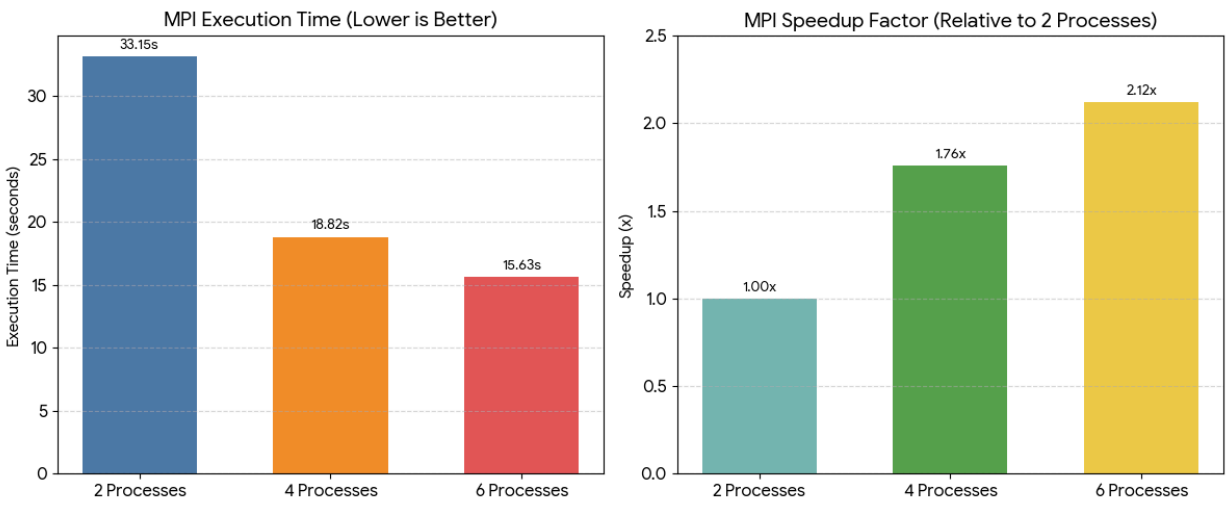
We use mountain.jpg as the test image for scalability evaluation.

OpenMP Scalability Analysis



We evaluated the OpenMP implementation's performance across 8, 16, and 32 cores. Initially, scaling from 8 to 16 cores resulted in a modest reduction in execution time (from **121.76s** to **112.1s**). However, a dramatic performance breakthrough occurred at **32 cores**, where execution time plummeted to **20.08s**. This significant, super-linear speedup suggests that at 32 threads, the system reached a critical hardware threshold—likely allowing the active working set of the image to fit entirely within the aggregate CPU cache—thereby eliminating memory bandwidth bottlenecks and maximizing parallel efficiency.

MPI Scalability Analysis

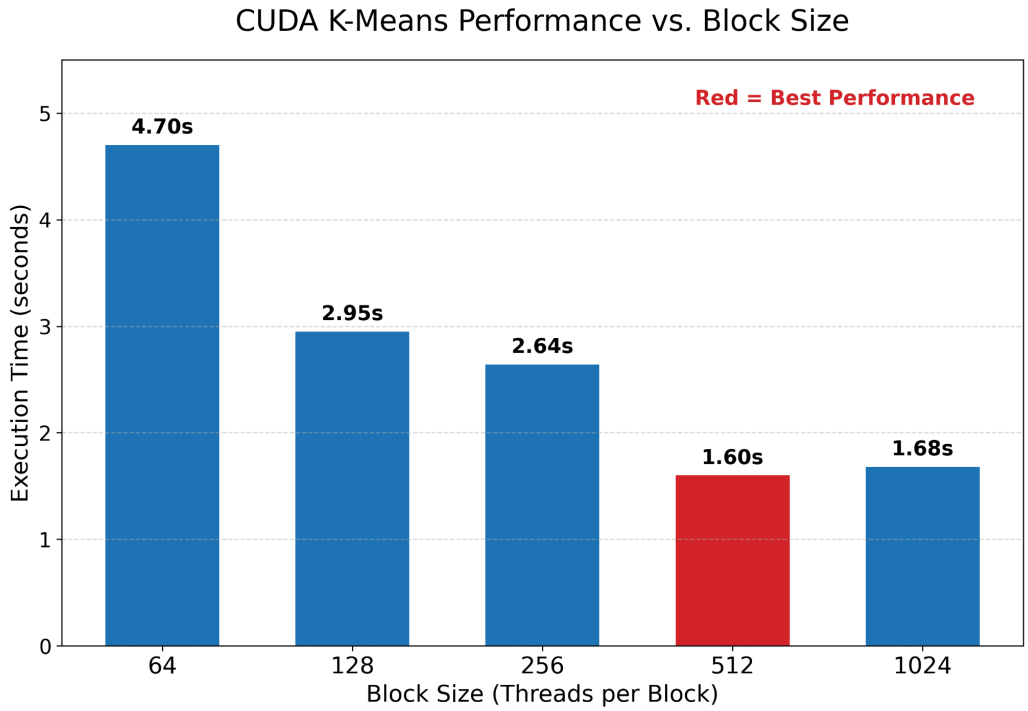


To evaluate the scalability of the MPI implementation within a shared-memory node, we measured the execution time across varying process counts (N_p={2, 4, 6}), keeping the problem size constant.

As illustrated in the figure, the execution time significantly decreases as the number of processes increases, dropping from **33.15s** (N_p=2) to **18.82s** (N_p=4) and finally **15.63s** (N_p=6). When calculating speedup relative to the 2-process baseline, we observed a **1.76x speedup** with 4 processes. This indicates a high parallel efficiency of approximately **88%** for the transition from 2 to 4 processes, demonstrating that the overhead of MPI communication (scatter/gather/reduce) is well-managed within the local memory interconnect.

However, scaling further to 6 processes yielded a diminishing return (speedup of **2.12x** vs. baseline), suggesting that as the sub-problem size per process shrinks, the constant overhead of synchronization and process management begins to outweigh the benefits of additional computational power. This trend is consistent with **Amdahl's Law** for strong scaling scenarios.

CUDA Block Size Scalability



The experimental results identify **512 threads** as the optimal configuration (1.60s), achieving the architectural "sweet spot" that effectively balances high warp occupancy with low overhead. The performance variation is driven by two main factors:

1. **Memory Bandwidth Saturation (Small Blocks)**: The 64-thread configuration performed the worst (4.70s). Using such a small block size forces the GPU to launch significantly more blocks. This results in a massive increase in partial sums being written to global memory, causing a bottleneck in memory bandwidth.
2. **Synchronization Latency (Large Blocks)**: The 1024-thread configuration (1.68s) was slightly slower than 512. Since all threads in a block must wait for each other at a barrier (`__syncthreads()`), synchronizing 1024 threads takes longer than 512, as the entire block must wait for the slowest thread. Additionally, fully saturating the block pushes the hardware's register resources to the limit, reducing the scheduler's flexibility.

Conclusion

In this project, we successfully implemented and benchmarked the K-means image color quantization algorithm across four distinct computing architectures: Serial, OpenMP, MPI, and CUDA. Our results demonstrate that while the "Assignment Step" of K-means is naturally suited for parallel execution, the optimal choice of hardware depends heavily on the problem size and the balance between computation and communication.

The **CUDA implementation** proved to be the superior solution for high-resolution images, achieving a massive **~65x speedup** by fully saturating the GPU's throughput. However, our analysis of smaller datasets (e.g., `bird.png`) revealed a critical insight: **parallelism is not free**. For small workloads, the fixed overheads of PCIe data transfer (CUDA) and message passing (MPI) outweighed the computational gains, allowing the low-latency OpenMP implementation to outperform the more complex architectures. Ultimately, this study highlights that achieving peak performance requires not just raw parallel power, but a strategic selection of architecture that aligns with the data scale and memory hierarchy requirements.

Work Distribution

ID	Name	Work
r13921093	LI, YEN-CHANG	MPI
r14921a15	HSIAO, YU-CHIEN	CUDA
t14902205	NATTHAPAT RATTHANARAK	OpenMP